



BOTHO
UNIVERSITY
EXCELLENCE | LEADERSHIP | INNOVATION

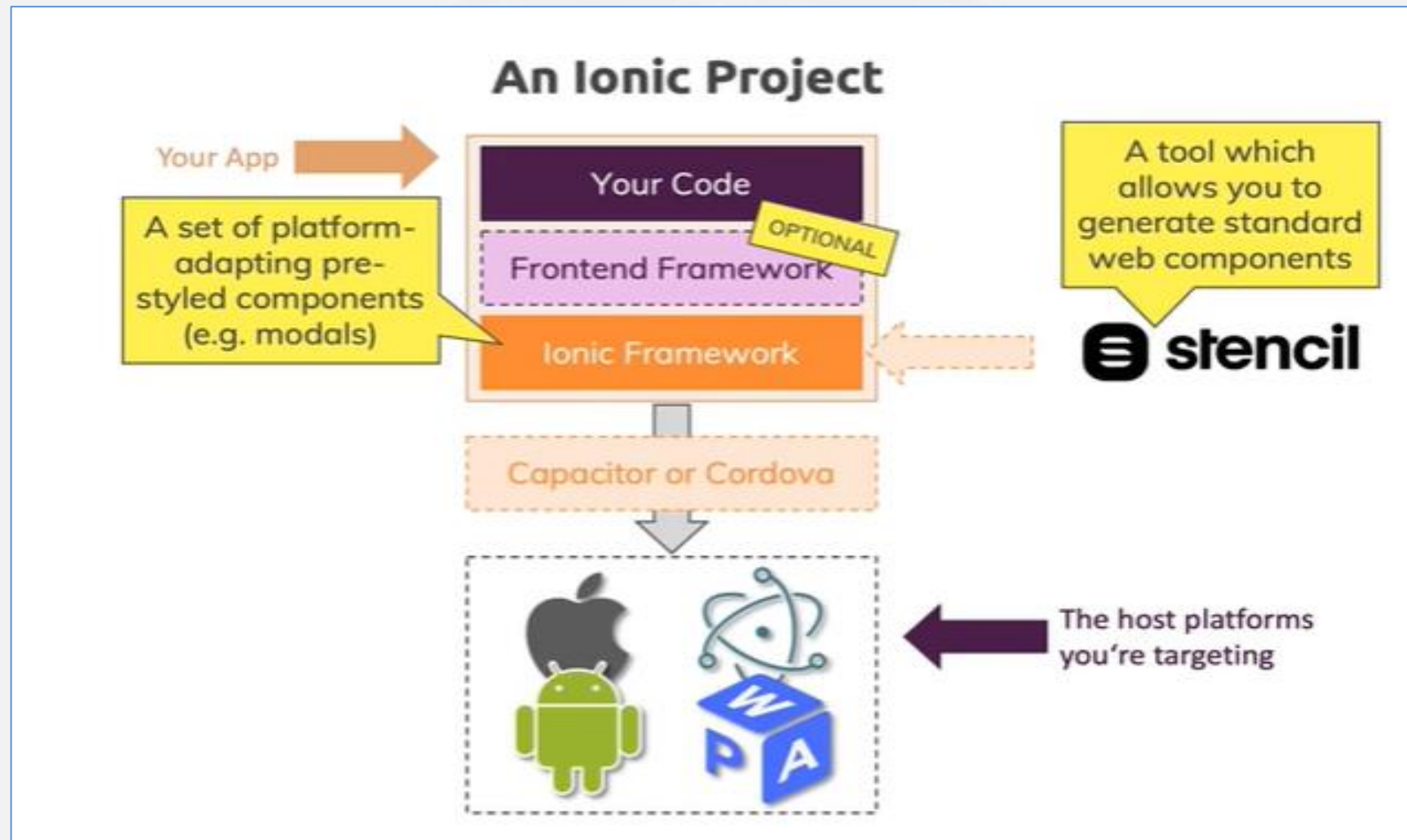
Introduction to Angular CLI

Objectives

In this session we will:

- Introduction to Angular

Introduction to Angular



Introduction to Angular Cont...

- **What is Angular?**

- A development environment built on **Typescript**.
- It is a component-based framework for building scalable web applications.
- It includes a collection of libraries that cover a variety of features such as routing, forms, management, client-server communication...
- It includes a suite of developer tools to help develop, build, test & update the code.

Introduction to Angular Cont...

- **Getting Started**

- To use Angular, install Node.js & node package manager (npm).
- To ensure that they are both installed correctly, issue the two commands: “*node -v*” & “*npm -v*”.
- Angular CLI allow users to create & manage projects from the command line. To install the CLI, run the command: “*npm install -g @angular/cli*”
- To check if Angular is correctly installed, issue the command: “*ng --version*”

```
Microsoft Windows [Version 10.0.19041.1110]  
(c) Microsoft Corporation. All rights reserved.
```

```
C:\Users\sehlabakan>ng --version
```

The logo for Angular CLI, featuring the word "Angular" in a stylized, outlined font, followed by "CLI" in a similar outlined font.

```
Angular CLI: 12.2.2  
Node: 14.17.5  
Package Manager: npm 6.14.14  
OS: win32 x64
```

Introduction to Angular Cont...

- **First Angular Project**

- Open up the new terminal & run the “*ng new*” command to create a new project: “*ng new angular-hello-world*”.
- This will run for while, but once it finishes, you will see a success message: “*Project ‘angular-hello-world’ successfully created*”.
- Navigating the project folder, you will see the following file structure:

```
4  |— README.md           // an useful README
5  |— .angular-cli.json   // angular-cli configuration file
6  |— e2e/               // end to end tests
7  |— karma.conf.js      // unit test configuration
8  |— node_modules/      // installed dependencies
9  |— package.json       // npm configuration
10 |— protractor.conf.js  // e2e test configuration
11 |— src/               // application source
12 └— tslint.json        // linter config file
```


Introduction to Angular Cont...

- The of interest is “src”, we’ll put our custom application code in here.

```
4  |-- app/
5  |   |-- app.component.css
6  |   |-- app.component.html
7  |   |-- app.component.spec.ts
8  |   |-- app.component.ts
9  |   |-- app.module.ts
10 |-- assets/
11 |-- environments/
12 |   |-- environment.prod.ts
13 |   |-- environment.ts
14 |-- favicon.ico
15 |-- index.html
16 |-- main.ts
17 |-- polyfills.ts
18 |-- styles.css
19 |-- test.ts
20 |-- tsconfig.json
```

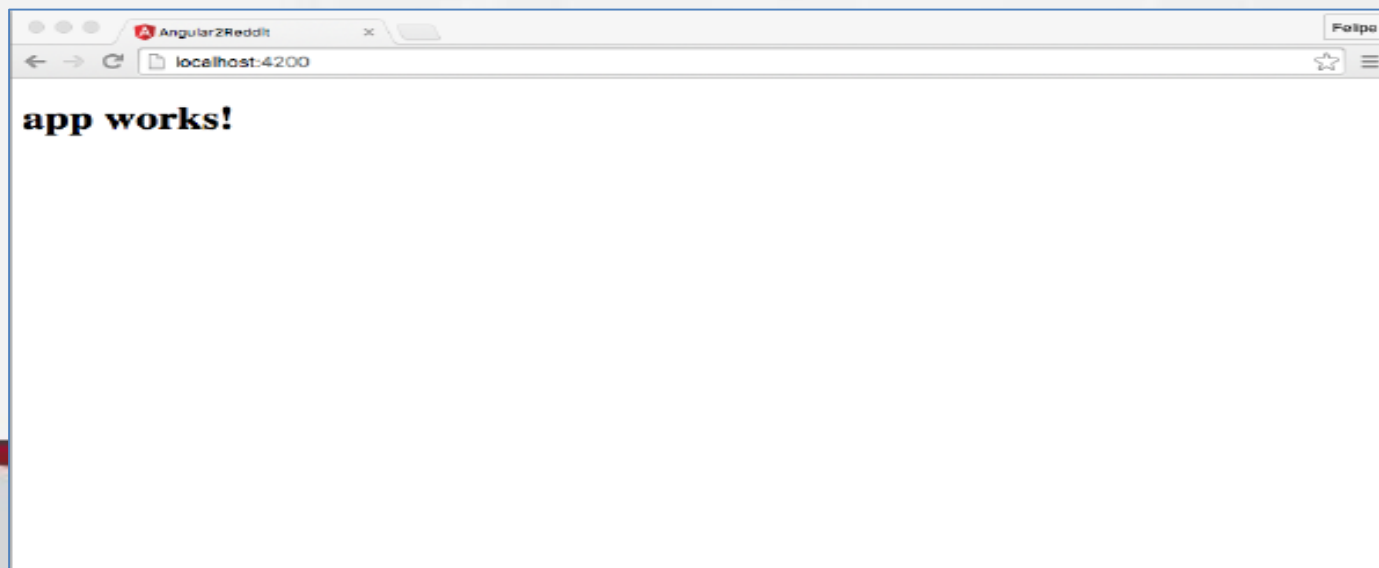
Introduction to Angular Cont...

- **Running the application**

- Angular CLI has a built in HTTP server that we can use to run our app.

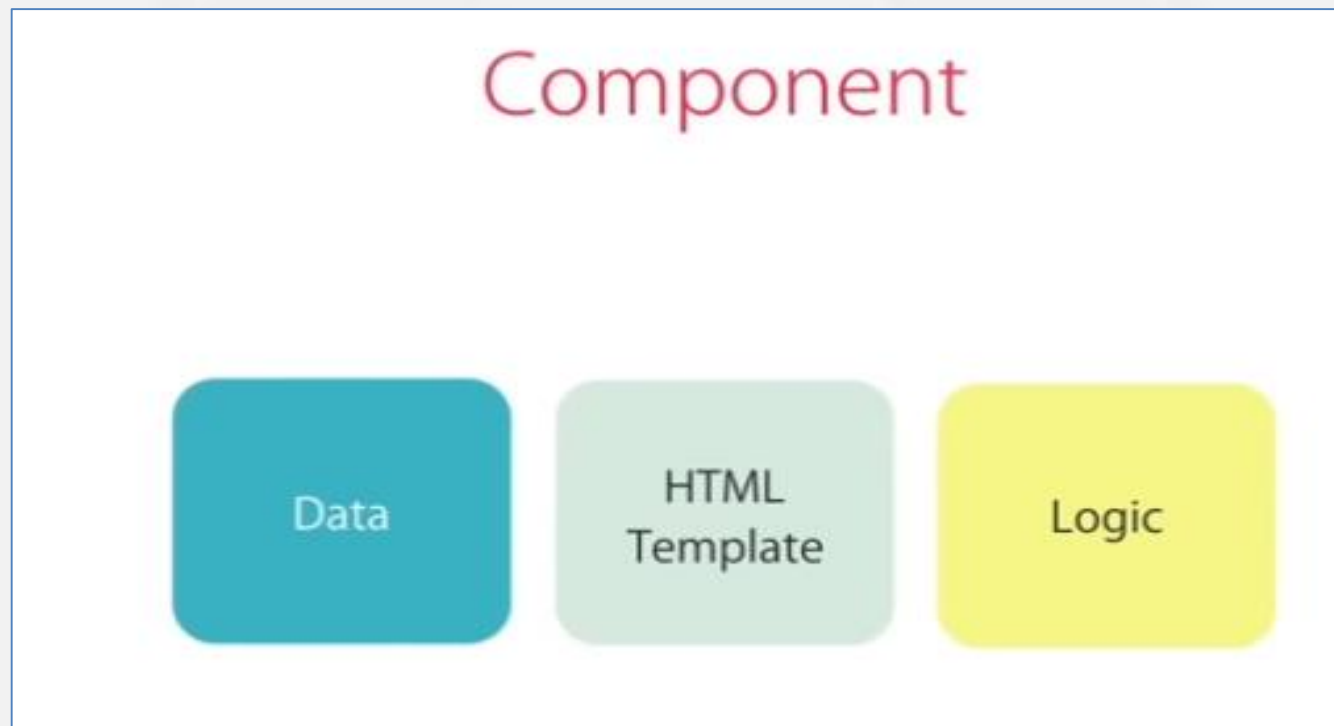
```
1 $ cd angular-hello-world
2 $ ng serve
3 ** NG Live Development Server is running on http://localhost:4200. **
4 // ...
5 // a bunch of other messages
6 // ...
7 Compiled successfully.
```

- Our app is now running on localhost port 4200. open the browser & visit, <http://localhost:4200>.

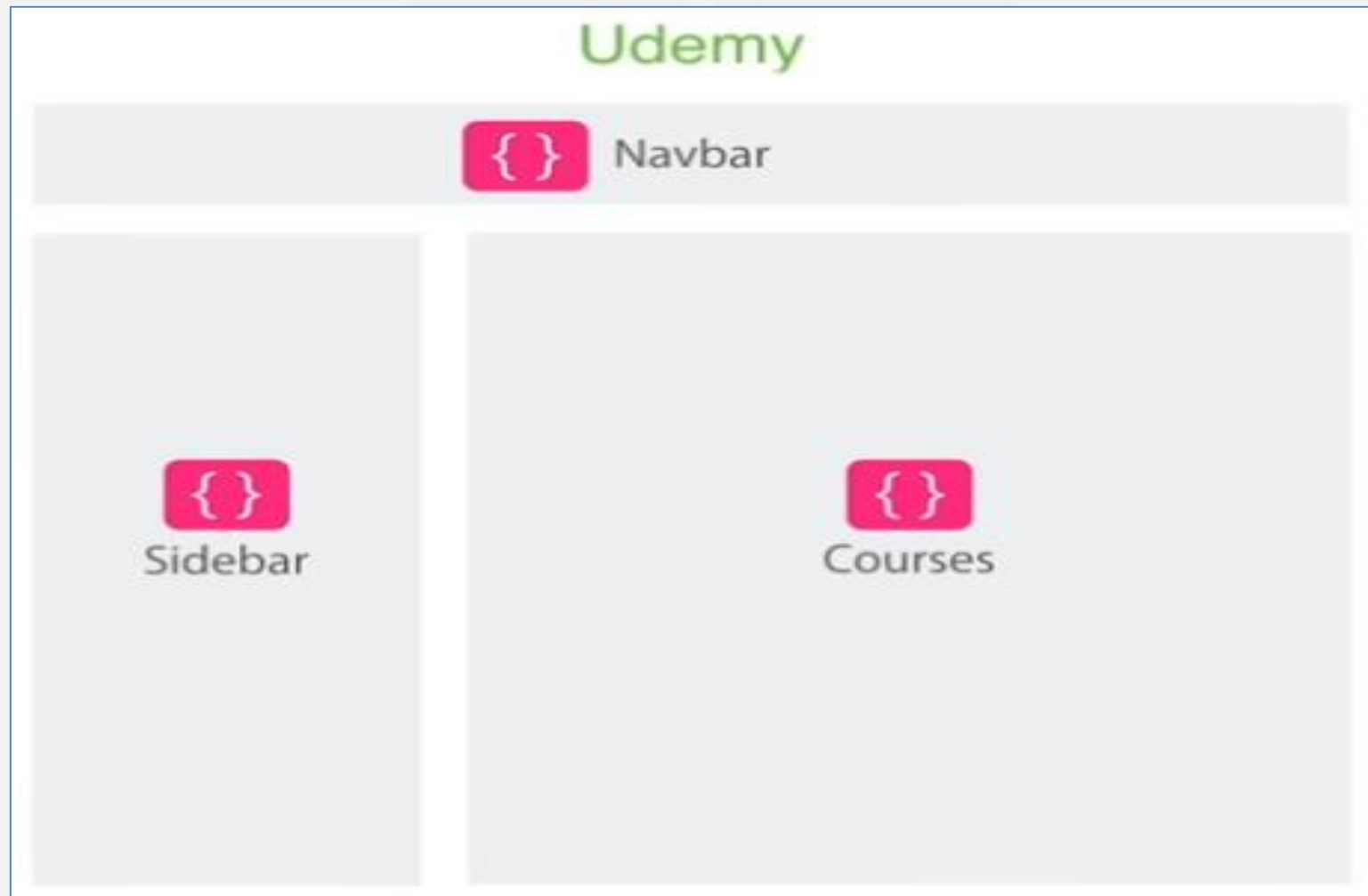


Introduction to Angular Cont...

- **Creating a Component**
 - Components are the building blocks that compose an application

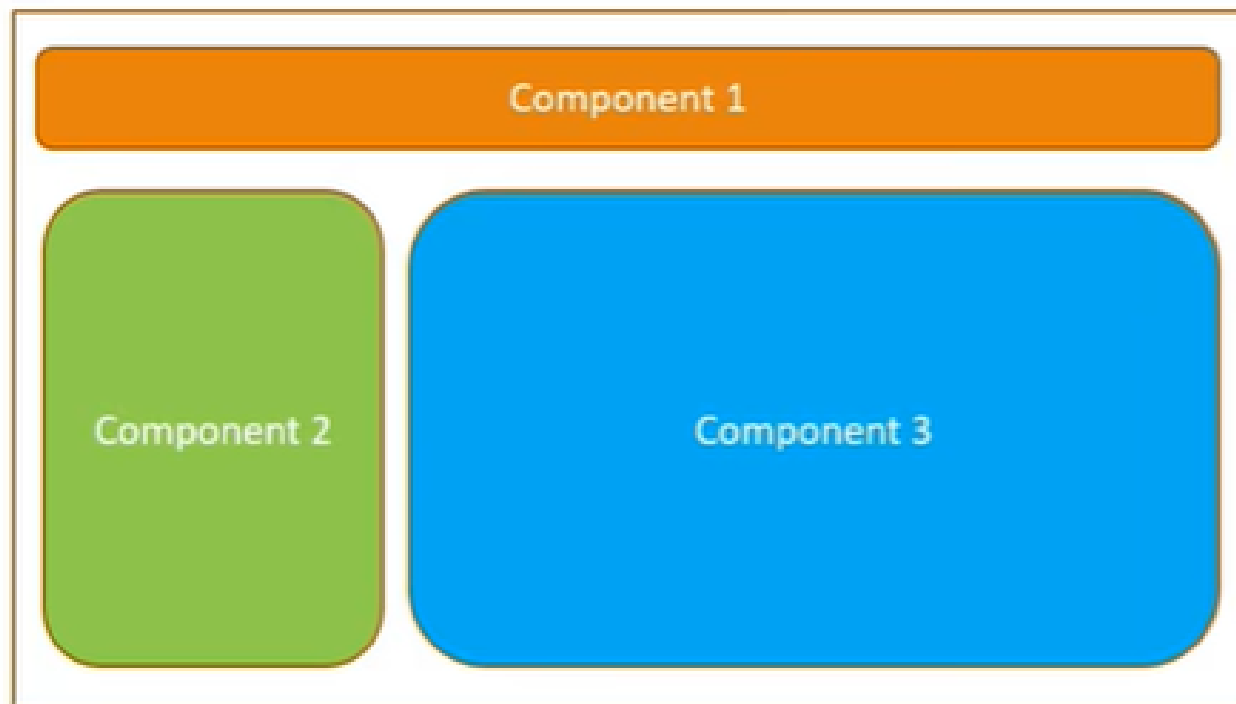


Introduction to Angular Cont...



Introduction to Angular Cont...

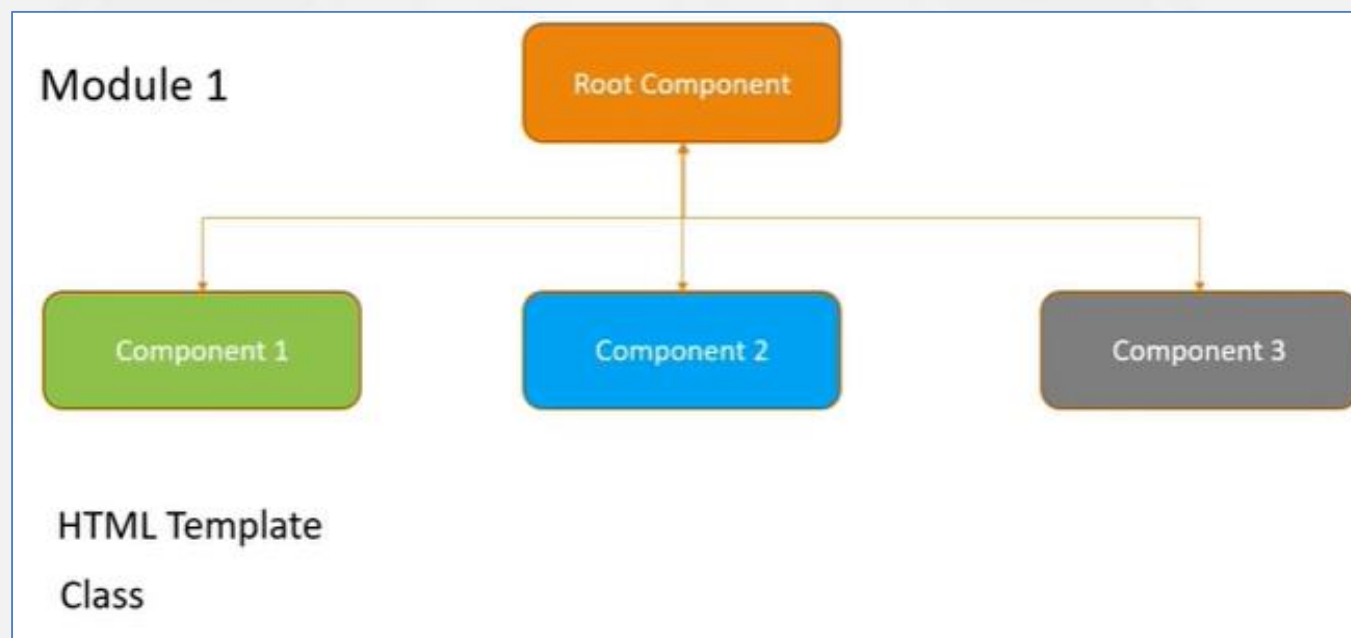
Components



Root Component
AppComponent

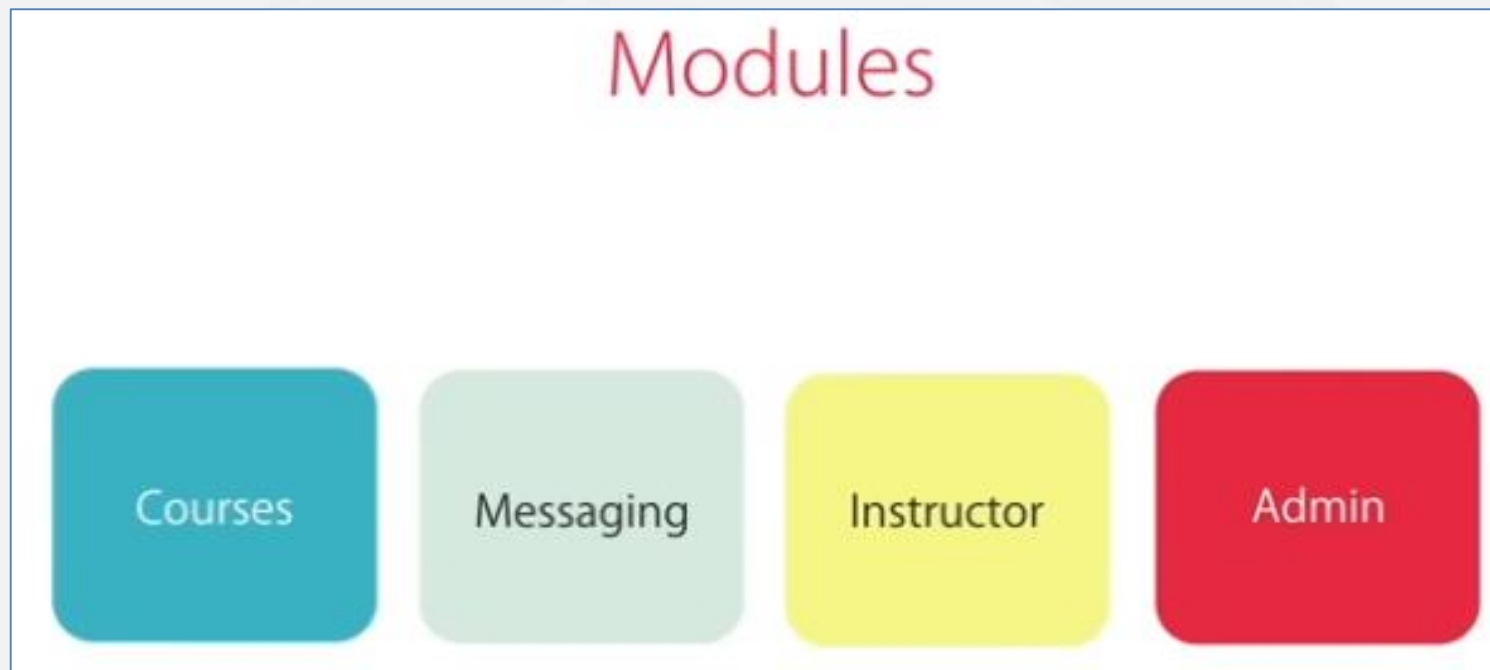
Introduction to Angular Cont...

- **Module**
 - A container for a group of related components.
 - Every angular app has atleast one module called AppModule.



Introduction to Angular Cont...

- Udemy Sample



Introduction to Angular Cont...

- **Creating Component**
 - Create a component
 - Register it in a module
 - Add an element in HTML markup

Introduction to Angular Cont...

- To create a new component using Angular CLI, use the generate command; “ng generate component hello-world”

```
1 $ ng generate component hello-world
2 installing component
3   create src/app/hello-world/hello-world.component.css
4   create src/app/hello-world/hello-world.component.html
5   create src/app/hello-world/hello-world.component.spec.ts
6   create src/app/hello-world/hello-world.component.ts
```

- A basic component has two parts:
 - A component decorator
 - A component definition class

```
code/first-app/angular-hello-world/src/app/hello-world/hello-world.component.ts
1  import { Component, OnInit } from '@angular/core';
2
3  @Component({
4    selector: 'app-hello-world',
5    templateUrl: './hello-world.component.html',
6    styleUrls: ['./hello-world.component.css']
7  })
8  export class HelloWorldComponent implements OnInit {
9
10     constructor() { }
11
12     ngOnInit() {
13     }
14
15 }
```

Introduction to Angular Cont...

- **Component decorators, templates & styling**

- After importing our dependencies, we are declaring the component.
- Decorators are **metadata added to the code**. When we use `@Component` on the `HelloWorld` class, we are decorating `HelloWorld` as a component.
- The idea is to use this component in our markup by using a `<app-hello-world>` tag.
- To achieve this, we configure the selector as “app-hello-world”

```
1 @Component({
2   selector: 'app-hello-world'
3   // ... more here
4 })
```

- In our component we are specifying a `templateUrl` of “./hello-world.component.html”.
- This means we will load our template from the file “hello-world.component.html”.
- Alternatively, a template can be added using the `template` key in our `@component` object.

```
1 @Component({
2   selector: 'app-hello-world',
3   template: `
4     <p>
5       hello-world works inline!
6     </p>
7   `
8 })
```

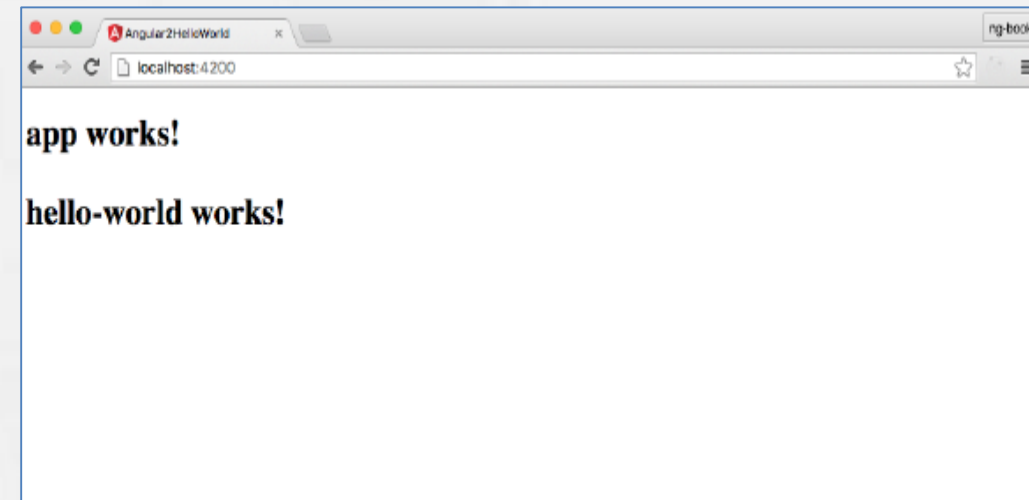
Introduction to Angular Cont...

```
1 styleUrls: ['./hello-world.component.css']
```

- This line tells us that we are using the CSS code in the file “hello-world.component.css”.
- **Loading the Component**
 - Now that we have our 1st component code filled out, how do we load it in our page?
 - Add the component tag to a template that is already being rendered & save.

code/first-app/angular-hello-world/src/app/app.component.html

```
1 <h1>
2   {{title}}
3
4   <app-hello-world></app-hello-world>
5 </h1>
```



Introduction to Angular Cont...

- **Adding Data to the Component**

- Lets create a component that will show a user's name.
- Use the command: “ng generate component user-item”
- Add our “app-user-item” tag to app.component.html.

```
code/first-app/angular-hello-world/src/app/app.component.html
1 <h1>
2   {{title}}
3
4   <app-hello-world></app-hello-world>
5
6   <app-user-item></app-user-item>
7 </h1>
```

```
code/first-app/angular-hello-world/src/app/user-item/user-item.component.ts
8 export class UserItemComponent implements OnInit {
9   name: string; // <-- added name property
10
11   constructor() {
12     this.name = 'Felipe'; // set the name
13   }
14
15   ngOnInit() {
16   }
17
18 }
```

- We want our `UserItemComponent` to show the name of a particular user.
- Introduce *name* as a new property of our component class. By having *name* property we will be able to reuse this component for different users.

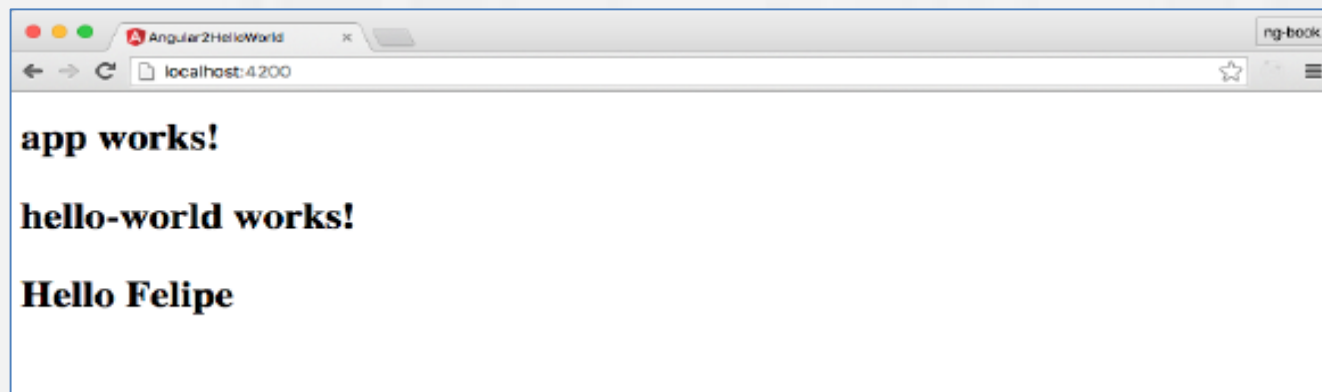
Introduction to Angular Cont...

- **Rendering the template**

- When we have a property on a component, we can show that value in our template by using curly brackets `{{}}` (interpolation) to display the value of the variable.

```
code/first-app/angular-hello-world/src/app/user-item/user-item.component.html
```

```
1 <p>
2   Hello {{ name }}
3 </p>
```



Introduction to Angular Cont...

- **Working with arrays**

- Now we are able to say “Hello” to a single name, but what if we want to say “Hello” to a collection of names?
- In Angular we can iterate over a list of objects in our template using the syntax `*ngFor`.
- Let’s create a new component that will render a *list* of users: “`ng generate component user-list`”
- And let’s replace our `<app-user-item>` tag with `<app-user-list>` in our `app.component.html` file.
- let’s add a `names` property to this `UserListComponent`.

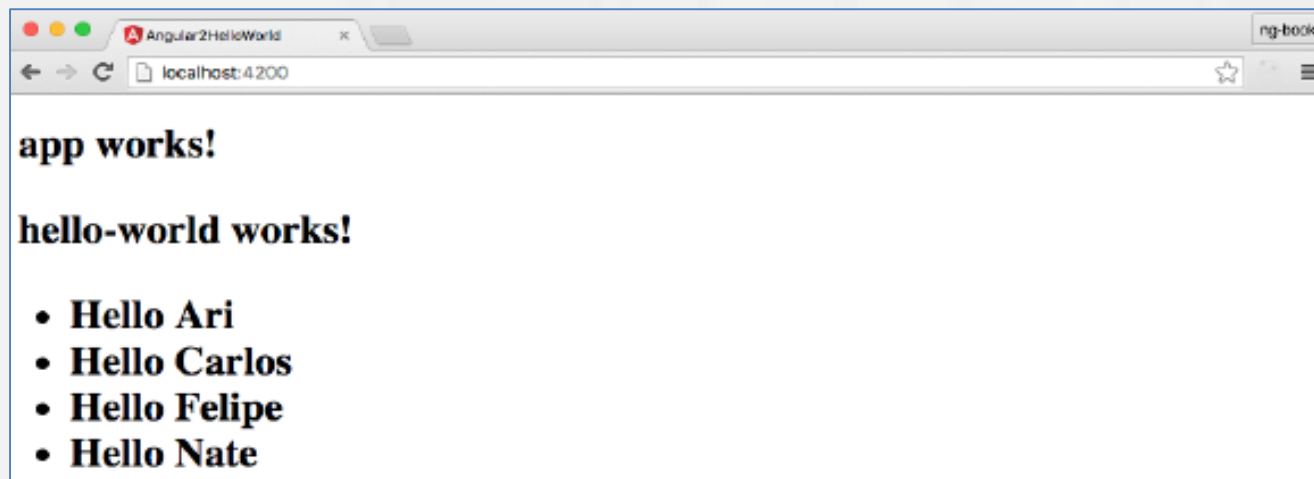
```
code/first-app/angular-hello-world/src/app/user-list/user-list.component.ts
8  export class UserListComponent implements OnInit {
9      names: string[];
10
11      constructor() {
12          this.names = ['Ari', 'Carlos', 'Felipe', 'Nate'];
13      }
14
15      ngOnInit() {
16      }
17
18  }
```


Introduction to Angular Cont...

- Now lets use *ngFor to iterate over a list of items and generate a new tag for each one.

code/first-app/angular-hello-world/src/app/user-list/user-list.component.html

```
1 <ul>
2   <li *ngFor="let name of names">Hello {{ name }}</li>
3 </ul>
```



Introduction to Angular Cont...

- **Using user Item Component as a child Component**
 - We should let our `UserItemComponent` specify the template (and functionality) of each item in the list.
 - Our `UserItemComponent` specifies the selector `app-user-item` - let's add that tag to our template:

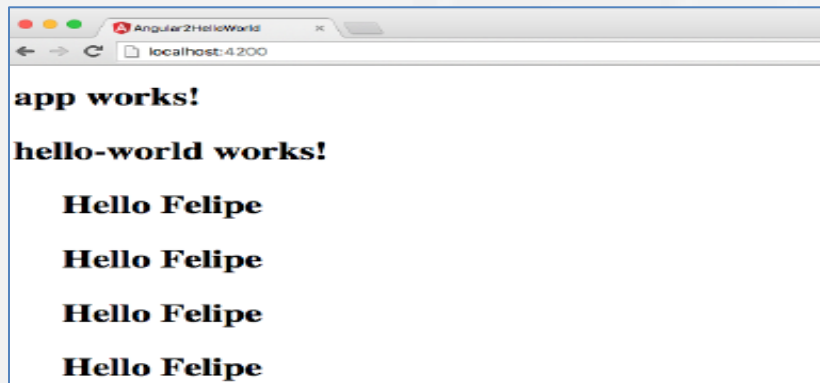
`code/first-app/angular-hello-world/src/app/user-list/user-list.component.html`

```
1 <ul>
2   <li *ngFor="let name of names">
3     <app-user-item></app-user-item>
4   </li>
5 </ul>
```

- Notice that we swapped out the text Hello and the name for the tag `app-user-item`. If we reload our browser, this is what we will see: (next page)

Introduction to Angular Cont...

- Definitely not what we wanted.
- We need a way to *pass data into the child component*. Angular provides a way to do this: the `@Input` decorator.
- This syntax allows us to pass in a value *from the parent template*.



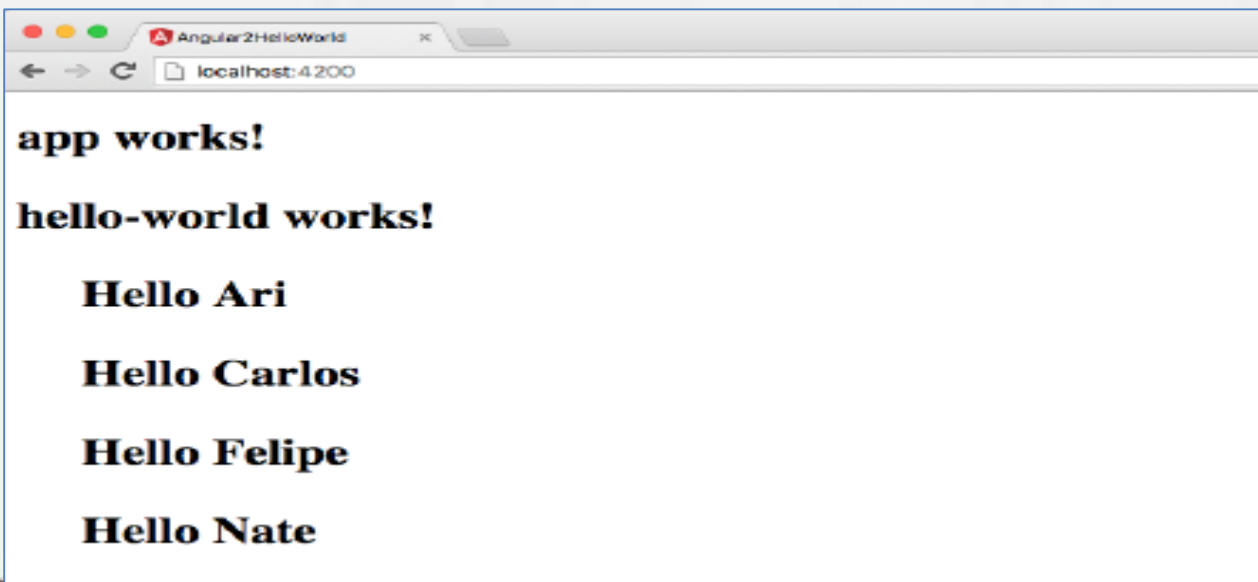
```
code/first-app/angular-hello-world/src/app/user-item/user-item.component.ts
1  import {
2    Component,
3    OnInit,
4    Input    // <--- added this
5  } from '@angular/core';
6
7  @Component({
8    selector: 'app-user-item',
9    templateUrl: './user-item.component.html',
10   styleUrls: ['./user-item.component.css']
11 })
12 export class UserItemComponent implements OnInit {
13   @Input() name: string; // <--- added Input annotation
14
15   constructor() {
16     // removed setting name
17   }
18
19   ngOnInit() {
20   }
21
22 }
```

Introduction to Angular Cont...

- To pass values to a component we use the *bracket* [] syntax in our template.

code/first-app/angular-hello-world/src/app/user-list/user-list.component.html

```
1 <ul>
2   <li *ngFor="let name of names">
3     <app-user-item [name]="name"></app-user-item>
4   </li>
5 </ul>
```



- **Summary**

In this session we have learnt about:

- Angular CLI basics